

EEEM068: Applied Machine Learning

Project: Training Vision Transformers for Knee Abnormality Classification

Project Overview

This project assesses your ability to apply the knowledge acquired in the lectures, laboratory sessions, and tutorials of EEEM068: Applied Machine Learning to a real-world medical imaging problem. You will design, implement, train, and evaluate a **Vision Transformer (ViT)** for knee MRI abnormality classification using the MRNet dataset collected at Stanford University Medical Center.

The project requires you to engage with several interconnected challenges that arise in practical deep learning for medical image analysis:

- Working with 3-D volumetric medical imaging data (MRI).
- Formulating a correct multi-label classification objective where a single patient can simultaneously present multiple co-occurring pathologies.
- Applying transfer learning from a self-supervised Vision Transformer pre-trained on natural images to a medical imaging domain.
- Evaluating model performance rigorously using metrics appropriate for imbalanced, multi-label datasets.
- Interpreting model behaviour through attention-based visualisations.
- Assessing model robustness when applied to images drawn from a different distribution than the training data.

You are expected to draw on the methods covered in Workshops 1–4 and adapt them appropriately for this setting.

Background

Vision Transformers

The Vision Transformer (ViT) architecture, introduced by Dosovitskiy et al. (2021), applies the Transformer architecture—originally developed for natural language processing—to image recognition. An image is divided into a fixed grid of non-overlapping patches (e.g. 16×16 pixels each), which are linearly projected into embedding vectors and treated as a sequence of tokens. A special [CLS] token is prepended to this sequence. The sequence is processed by multiple self-attention blocks, and the final representation of the [CLS] token is used for classification.

Key properties relevant to this project:

- **Self-attention:** Each token attends to all other tokens, capturing long-range spatial dependencies across the image without the locality bias of convolutional filters.
- **Attention maps:** The attention weights from the [CLS] token to image patch tokens encode where the model is “looking” when making its prediction. These weights can be extracted and visualised to produce class attention maps, providing a form of weakly-supervised localisation.
- **Scalability:** ViTs require large amounts of data to train from scratch; in practice, one always begins from a pre-trained checkpoint.

For this project, you will use the **ViT-Small (ViT-S/16)** variant, which uses a patch size of 16, an embedding dimension of 384, 6 attention heads, and 12 transformer blocks.

Multi-label Classification

The MRNet dataset provides three *independent* binary labels per exam: **abnormal**, **acl** (ACL tear), and **meniscus** (meniscal tear). These labels are **not mutually exclusive**: a single knee MRI can be simultaneously positive for multiple conditions. The published MRNet statistics confirm that 37.1% of

exams have meniscal tears, 23.3% have ACL tears, and 80.6% are abnormal—and a non-trivial proportion are positive for more than one condition simultaneously.

It is therefore **incorrect** to treat this as a single-label multi-class problem by assigning one exclusive class per exam. Doing so would silently discard co-occurring conditions, preventing the model from ever producing a correct prediction for the suppressed condition. The correct formulation is **multi-label binary classification**: three independent binary outputs, one per condition, each trained with a binary cross-entropy objective. All design decisions—including your loss function, evaluation metrics, and prediction thresholding—must reflect this multi-label nature.

Dataset

MRNet Dataset

The MRNet dataset consists of 1,370 knee MRI examinations performed at the Stanford University Medical Center [?]. Each examination is stored as a three-dimensional numpy array (`.npz`) of shape $(\text{num_slices}, H, W)$, representing a volumetric MRI scan acquired from one of three anatomical planes. The dataset is divided into:

- **Training set:** 1,130 examinations.
- **Validation set:** 120 examinations (used as your test set; do not use this for hyperparameter selection—see Section ??).

The dataset is available on Kaggle:

<https://www.kaggle.com/datasets/cjinny/mrnet-v1?resource=download>

Directory Structure

After downloading and extracting, the dataset has the following layout:

```
MRNet-v1/
├── train/
│   ├── axial/      0000.npy, 0001.npy, ...
│   ├── coronal/    0000.npy, 0001.npy, ...
│   └── sagittal/    0000.npy, 0001.npy, ...
├── valid/
│   ├── axial/      ...
│   ├── coronal/    ...
│   └── sagittal/    ...
├── train-abnormal.csv
├── train-acl.csv
├── train-meniscus.csv
├── valid-abnormal.csv
├── valid-acl.csv
└── valid-meniscus.csv
```

Each CSV file has two columns (no header): `exam_id` and `label` (0 or 1). The filename stem (e.g. 0042) identifies the corresponding `.npz` volume.

Anatomical Planes

Each examination is available in three anatomical planes:

- **Sagittal:** Side view; primary plane for ACL assessment.
- **Coronal:** Front-facing view.
- **Axial:** Cross-sectional (top-down) view.

As a minimum, you must implement support for at least one plane. Investigating the effect of plane selection or plane fusion is encouraged and may be rewarded with extra credit (Section).

Volumetric Data

Each `.npy` file is a 3-D array with shape $(\text{num_slices}, H, W)$ where $H = W = 256$ in most cases. The number of slices varies across examinations (typically between 20 and 50). You must implement a strategy for converting 3-D volumes into 2-D inputs suitable for the ViT. Possible strategies include, but are not limited to: Your choice of strategy must be *justified* and *evaluated* in the report.

Implementation Requirements

The following subsections describe what you are required to implement. Each requirement maps directly to a marking criterion (Section). You may use any deep learning framework, but **PyTorch** is strongly recommended, as it is used throughout all workshops.

1. Dataset Loading and Pre-processing
2. Data Augmentation
3. Model Architecture
4. Training
5. Evaluation
6. Class Attention Map Visualisation
7. Out-of-Distribution Inference

After training and evaluating on the MRNet dataset, collect at least six publicly available knee MRI images from the internet (search terms such as “ACL tear sagittal plane MRI”, “meniscal tear MRI”, “normal knee MRI sagittal” will yield suitable images). You must:

- a. Pre-process these images using the same normalisation and resize pipeline used for the MRNet validation set.
- b. Run inference with the trained model and report per-condition sigmoid probabilities and binary predictions for each image.
- c. Compare these results quantitatively and qualitatively against the model’s performance on the MRNet validation set.
- d. Provide a **detailed analysis** of why the predictions for out-of-distribution images may differ from in-distribution predictions. Your analysis should address factors such as scanner manufacturer and protocol differences, image resolution, field of view, and the absence of DICOM pre-processing.

Submission

Submit a single `.zip` archive to SurreyLearn containing:

1. **Code directory** (`code/`): All Python source files required to reproduce your results, plus a `README.md` file describing the exact commands needed to install dependencies, train the model, and run evaluation. Your code must be runnable on a clean Python environment using only packages declared in your `requirements.txt`. Do **not** include the dataset or model checkpoints in the submission (they are too large).
2. **Report** (`report.pdf`): A written report in **IEEE double-column format**, not exceeding **5 pages** of main text. Additional pages (beyond page 5) are permitted only for figures and references. The main text of the report must be self-contained within 5 pages; do not place substantive discussion in appendix pages.

Report structure. Your report must include the following sections. The suggested word counts are approximate guides; marks are awarded for quality and depth, not length.

1. **Abstract** (~150 words): Summarise the problem, your approach, and the key numerical results.
2. **Introduction** (~300 words): Motivate the clinical importance of knee abnormality detection; describe the limitations of conventional CNN-based approaches that ViTs address; state the specific contributions of your work.
3. **Related Work** (~400 words, minimum **5** cited papers): Review the relevant literature, covering at least: (i) the original ViT paper, (ii) one data-efficient or self-supervised ViT pre-training method, (iii) the MRNet paper or a comparable medical MRI classification work, (iv) one paper on attention visualisation for transformers.
4. **Methodology** (~600 words): Describe your complete pipeline including dataset loading, label formulation, augmentation, model architecture, loss function, optimiser, and learning rate schedule. Include a schematic or diagram of the overall system. Justify every design decision.
5. **Experiments and Results** (~700 words): Present all quantitative results in clearly labelled tables and figures. Include the baseline comparison, per-condition metrics, confusion matrices, attention map figures, training curves, hyperparameter ablation, and OOD inference results. Every figure and table **must** be referenced by number in the text with a discussion of what it shows.
6. **Conclusion and Future Work** (~200 words): Summarise your findings; identify the main limitations of your approach; propose at least two concrete directions for future work.

Figures. All figures must include a descriptive caption. Figures without captions, or figures not referenced in the main text, will receive no marks.

Marking Criteria

The project is assessed across three dimensions. The total available mark is 100 points.

A. Report Quality [50 marks]

Whether the results are well-presented, correctly interpreted, and thoroughly discussed.

Component	Marks
Abstract	5
Introduction	10
Related Work (must cite ≥ 5 papers)	15
Methodology	25
Experiments & Results	35
Conclusion & Future Work	10
Total	100

Marks within each component are awarded according to the following rubric:

- **Abstract** [5]: Concisely states the problem, method, and key numerical result. Vague or generic abstracts receive $\leq 2/5$.
- **Introduction** [10]: Provides clinical motivation; correctly identifies the multi-label nature of the problem and why it differs from standard multi-class classification; clearly states the paper's contributions.
- **Related Work** [15]: At least 5 relevant papers are cited and discussed substantively (not merely listed). The discussion situates the proposed approach relative to the literature. Papers cited without discussion receive zero credit.
- **Methodology** [25]: All design decisions are described precisely and justified. The multi-label formulation, loss function, positive-class weighting, differential learning rates, and slice-sampling strategy are explicitly addressed. A pipeline diagram is provided. Missing or unjustified design decisions attract deductions proportional to their importance.

- **Experiments & Results** [35]: All required metrics are reported (per-condition accuracy, precision, recall, F1, AUC-ROC, confusion matrices). Baseline comparison is provided. Hyperparameter ablation is presented. TensorBoard training curves are included. Attention maps are discussed qualitatively. OOD analysis is provided with causal explanation. Results without discussion receive at most half the available marks for that sub-component.
- **Conclusion** [10]: Accurately summarises the main findings; identifies genuine limitations; proposes concrete and feasible future directions.

B. Functionality [30 marks]

Whether the submitted code implements all required components and produces correct outputs.

Partial credit is awarded for partially correct implementations. Implementations that treat the problem as single-label multi-class (e.g. using `CrossEntropyLoss` or `argmax` for prediction) will receive only half of the mark of this section

C. Code Quality [20 marks]

Quality, clarity, efficiency, and documentation of the submitted code.

Extra Credit

Extra credit of up to **15 additional marks** is available for implementations that go beyond the base requirements. Extra-credit work for the first two, must be described and evaluated in the report; implementation alone without analysis will receive no extra credit.

- **Multi-plane fusion** [up to 3 marks]: Train separate models for each anatomical plane (sagittal, coronal, axial) and combine their predictions (e.g. by averaging sigmoid outputs). Report whether fusion improves per-condition AUC-ROC relative to the best single-plane model, and analyse which plane contributes most to each condition.
- **Feature space visualisation** [up to 3 marks]: Extract the [CLS]-token embeddings for all validation images, project them into 2-D using t-SNE or UMAP, and visualise the resulting scatter plot colour-coded by condition label. Discuss the degree of per-condition clustering and relate it to the model's classification performance.
- **Weights & Biases integration** [up to 3 marks]: Integrate the W&B logging framework to track all experiments, produce a parallel coordinates plot over the hyperparameter search, and include a link to a public W&B project in the report.
- **Systematic error analysis** [up to 3 marks]: Identify the validation images for which the model is most confidently incorrect (highest loss). Display these images with their attention maps and provide a reasoned hypothesis for each failure.
- **Project-style implementation** [up to 3 marks]: Rather than submitting a monolithic Colab notebook where all code is written in a single sequential script, structure your submission as a proper software project. Full marks for this item require all of the following:
 - Separate, clearly named Python modules for distinct concerns (e.g. `dataset.py`, `model.py`, `train.py`, `evaluate.py`).
 - A command-line entry point (e.g. `main.py` or `train.py`) with named arguments for all configurable hyperparameters, so that any experiment can be reproduced with a single shell command without editing source files.
 - A `requirements.txt` (or equivalent) that pins all dependencies to specific versions, enabling a clean-environment install with `pip install -r requirements.txt`.
 - A `README.md` at the project root providing exact commands to install dependencies, download the dataset, train the model, and run evaluation.